

Technická Dokumentace k Software Exposea - Systém pro snímání a automatické vyrovnávání předloh



Dávid Pukanec, Marek Šolony, Michal Hradiš

Tento dokument byl vytvořen s finanční podporou MK ČR v rámci programu **NAKI III program na podporu aplikovaného výzkumu v oblasti národní a kulturní identity na léta 2023 až 2030**

v projektu Smart digilinka - Strojové učení pro digitalizaci tištěného dědictví.

Číslo a název projektu:

DH23P03OVV066	Smart digilinka - Strojové učení pro digitalizaci tištěného dědictví
---------------	--

Název a popis dílčího výstupu:

Exposea - Systém pro snímání a automatické vyrovnávání předloh
Tento dokument popisuje funkčnost a použití software Exposea, nástroj pro zarovnání a spojení částečných fotografií velkého dokumentu do jednoho obrazu ve vysokém rozlišení, který zachovává detaily, odstraňuje viditelné přechody a sjednocuje osvětlení mezi jednotlivými snímky.

Jazyk dokumentu

Angličtina

Organizace a řešitel

Vysoké učení technické v Brně	Ing. Michal Hradiš Ph.D.
-------------------------------	--------------------------

License

Software: BSD 3-Clause License

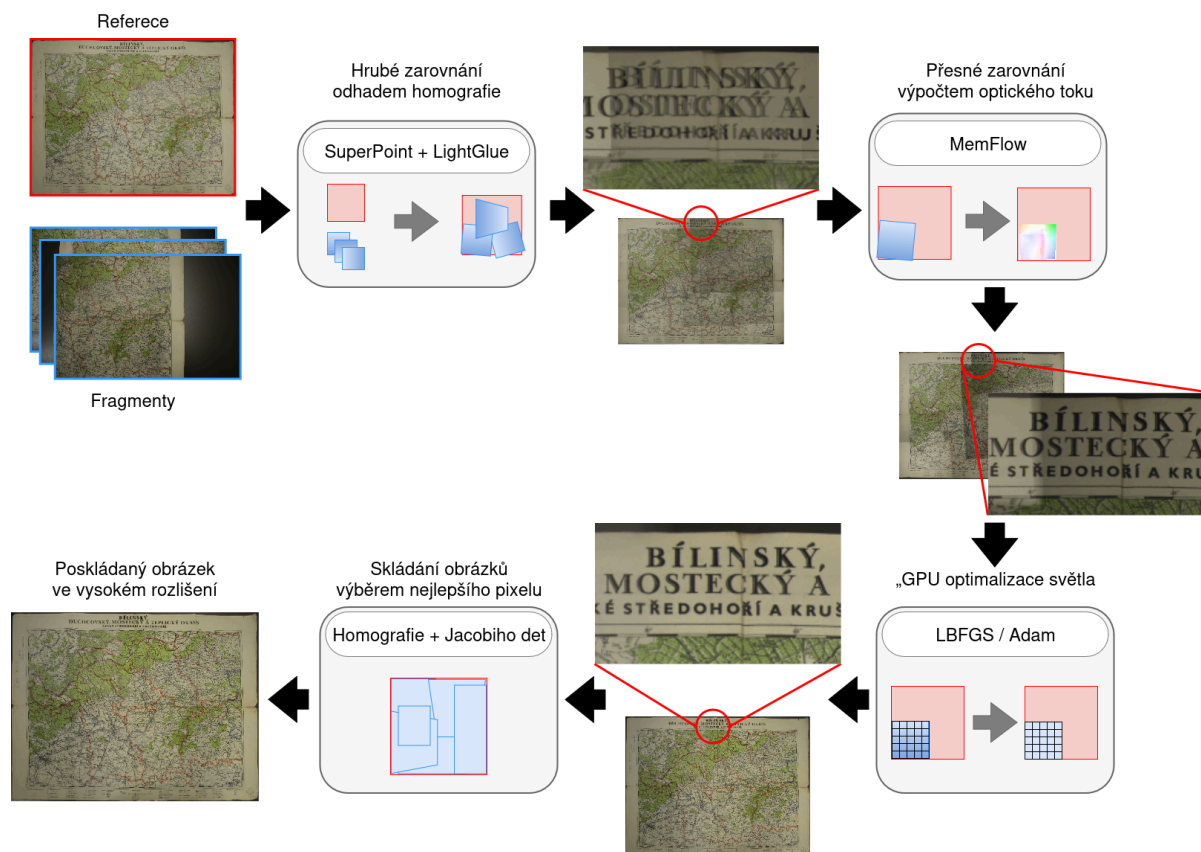
Overview

The main task of the application is to align and merge partial photographs of a large document into a single high-resolution image, preserving details, removing visible seams, and unifying lighting between individual photos. The application works with one reference photograph of the entire document and several detailed close-up shots (fragments) that capture its parts in higher detail.

The system is based on several deep learning methods for fragment alignment and GPU-accelerated lighting optimisation using the reference image. The entire application is designed to be fully configurable through configuration files, allowing users to adjust all system parameters. To minimise user input, only a few essential parameters need to be specified manually, while the remaining ones can be selected from predefined configuration presets.

Processing Pipeline

The processing pipeline is shown in the following image and the individual steps are briefly described in the following sections.



1. Estimation of coarse alignment

In this step, keypoints are detected in the images using the **SuperPoint** method, and their correspondences are found using **LightGlue**. Based on the estimated correspondences and the RANSAC method, a transformation is computed to approximately align each fragment to the reference image. This processing is performed at a reduced resolution for efficiency.

2. Estimation of precise alignment

The roughly aligned fragments are then aligned at the pixel level with respect to the reference image using the **MemFlow** optical flow method. This step is also carried out at reduced resolution due to computational demands, and the results are subsequently scaled to the desired resolution using interpolation.

3. Illumination optimization

Since close-up image acquisition often leads to lighting variations, these differences must be corrected. In the aligned fragments, illumination is adjusted to match the reference image using the **Adam** or **L-BFGS** optimizers. The fragment is divided into smaller regions due to memory constraints, and a parameter field of chosen dimensions is created, where each parameter represents a local illumination scale. Using an appropriate loss function—typically Mean Squared Error (MSE)—the parameters are estimated to best match the illumination of the reference image.

4. Composition of the final image

The aligned and illumination-corrected fragments are progressively inserted into the final composite image. From each fragment, the pixels that best preserve local details are selected. For this purpose, the Jacobian determinant computed from the transformation matrix obtained in the first step is used. The selected regions are smoothly blended along their edges to avoid visible seams. Finally, the resulting image is saved in JPEG2000 or TIFF format.

Availability

The software is available on the GitHub repository: <https://github.com/DCGM/Exposea>. The repository contains all the necessary code, along with instructions for installing the application, docker files, docker compose configuration, and a test example to verify that everything works properly.

Requirements overview

The entire application is written in Python 3.11, and therefore, we recommend using Python version 3.11 or higher. Additionally, a virtual environment with the necessary libraries must be created. The necessary packages are listed in the *requirements.txt* file. We do not guarantee that the program will work correctly with lower versions of Python or required libraries.

When installing via Docker, both Docker Engine and Docker Compose must be installed. For more information, please refer to <https://www.docker.com/get-started/>

The application requires a system equipped with a GPU supporting CUDA. When installing using Docker, it is also necessary to install the NVIDIA Container Toolkit <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>.

Hardware requirements are listed in the table below. The following processing times were measured on a computer equipped with an Intel i7-13700KF (13th generation) processor, an NVIDIA RTX 3090 graphics card with 24 GB of memory, and 120 GB of RAM:

Number of fragments	Output resolution	RAM consumption	Composition time
10	100 Mpx	27 GB	425s
10	300 Mpx	54 GB	996s
10	500 Mpx	81 GB	1655s
10	700 Mpx	110 GB	2339s
10	900 Mpx	120 GB	2835s

Installation

Two installation options are available:

1. **Command-line installation** using a pre-created virtual environment.
2. **Docker-based installation** using the provided Docker configuration.

Command-line installation

The procedure involves cloning the repository and installing the libraries required to run the program. We recommend cloning the newest release of Exposea.

```
$ git clone --branch v1.00-Akita https://github.com/DCGM/Exposea.git
$ cd Exposea
$ pip install --upgrade pip && pip install -r requirements.txt
$ sudo apt update
$ sudo apt install libopenjp2-tools
```

Docker-based installation

For installation using Docker, the Git repository containing the *Docker* directory must first be downloaded or cloned using git.

```
$ git clone https://github.com/DCGM/Exposea
```

Within the *Docker* directory, there is a configuration file named *docker-compose.yaml*, in which the paths to the input and output directories must be specified. The input directory contains the files for individual tasks, while the output directory stores the processed results.

By default, the configuration file is set to process the sample data located in the *test_data* directory and to save the output to the *output* directory.

```
volumes:
  - ../test_data:/data
  - ../output:/output
```

After adjusting the configuration, the Docker container must be built.

```
$ cd Exposea/docker
```

```
$ docker compose -f docker-compose.yaml build
```

Once the container has been created, it is necessary to choose whether to:

- Run the **service mode**, which continuously monitors the input directory and processes each newly added unprocessed file.

```
$ docker compose up exposea_service
```

- Perform a **single-run process**, which processes only one input file and then terminates.

```
$ docker compose up exposea
```

Run instructions

The Exposea application can be executed in two ways.

1. Command-line execution is suitable for processing a single task at a time. The input directory must have the required structure and contain the necessary files as specified below.

```
$ python register.py -i ./cesta/k/ufoze -o ./cesta/k/vystupu/
```

2. The second method involves running the application using Docker, either in the “**one-shot**” mode, which processes all unprocessed tasks from the input directory, or in the “**service**” mode, which periodically monitors the input directory and processes all new unprocessed tasks.

```
$ docker compose up exposea_service
```

Note: When running from the command line, the input path should point to a specific task directory (e.g., `/input/file/task1/`), whereas when using Docker, the path refers to the parent directory containing multiple task folders (e.g., `/input/file/`).

Input folder structure

Each directory representing a single document reconstruction task must contain a subdirectory named *images* with all input images, and a configuration file named *config.yaml*. The contents of this configuration file are described below.

Example of input directory structure:

```
|— task1_processing
|— task2_done
|— task3
|   |— images
|   |— config.yaml
|— task4_incomplete
|   |— images
|   |— config.yaml
```

Guidelines for Input Folders

When running the program through a Docker image, it is essential to ensure atomic operations on input directories. To achieve this, several suffixes are defined to indicate the current processing state of each directory. These suffixes inform both the program and the user about which operation is being performed or has been completed. This behaviour applies **only** when processing input directories using Docker.

Special suffixes and their meaning:

- **_incomplete** – Recommended for directories that are currently being copied into the input folder to prevent premature processing.
- **_processing** – Indicates that the directory is currently being processed. It should not be modified to avoid runtime errors.

- **_done** – Marks a directory that has already been processed; such directories are ignored by the program.

Any directory without these suffixes is considered suitable for processing.

Configuration file

Each task directory must contain a configuration file named *config.yaml*, which defines the parameters for running the application. Only a few mandatory parameters need to be specified, as listed below:

- **exp_name** – The name under which the output will be saved.
- **ref_name** – The name of the reference image located in the *images* folder.
- **preset_name** – The name of the preset configuration file (without the *.yaml* extension). Since the application can process images of various resolutions, different parameter sets are required for different resolutions. Exposea provides several predefined presets, one of which must be selected.
- **final_res** – The desired resolution of the final image in pixels, specified as *[height, width]*.
- **save_format** – The format in which the resulting image will be saved. Possible values: *jp2, j2k, tiff, tif*.
- **corner_points** – A list of corner coordinates in the format *[[width, height], ...]*.

For optimal performance, it is also recommended to specify:

- **relative_scale** – A multiplier indicating how much smaller the fragments are compared to the reference image. This value can be estimated based on the number of fragments required to cover the entire document. For example, if 4 images cover the document, the relative scale is 2; if 9 images are needed, the relative scale is 3, and so on. Although this value does not need to be exact, a more accurate estimate helps improve the reconstruction quality.

The list of currently available presets can be printed by running:

```
$ python register.py -p
```

Additional parameters

Exposea was developed with all parameters fully configurable. While it is not strictly necessary, users may modify any parameter defined in the *default.yaml* configuration file. This is possible because configuration parameters are applied hierarchically, with the following order of precedence: *default.yaml* < *preset.yaml* < *config.yaml*. We recommend specifying parameters in the **task-specific** *config.yaml* file, as these settings override both the preset and default configuration files.